

COMS W4156 Advanced Software Engineering (ASE)

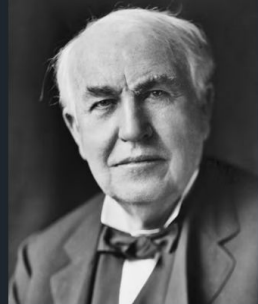
November 21, 2024

On this day ...



First Surviving Motion Picture

1890 Edison lab records the 1st surviving motion picture, "Monkeyshines No. 1", shot by William am Kennedy Dickson and William Heise [date disputed between June 1889 and November 21–27, 1890]



Inventor

Thomas Edison

Agenda

1. authentication/authorization demo: Shivam Chaturvedi and Garlic Bread team
2. MVC

as time permits:

3. test oracles
4. test suite quality
5. test generation
6. rest of semester



Shivam Chaturvedi and Garlic Bread team

Authentication and Authorization Demo

[leader slides](#)

[team slides](#)



okta

Agenda

1. authentication/authorization demo: Shivam Chaturvedi and Garlic Bread team

2. **MVC**

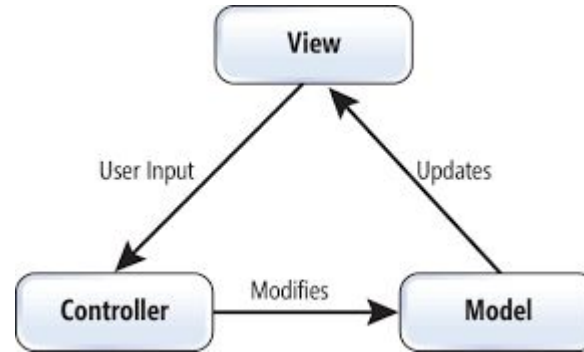
as time permits:

3. test oracles

4. test suite quality

5. test generation

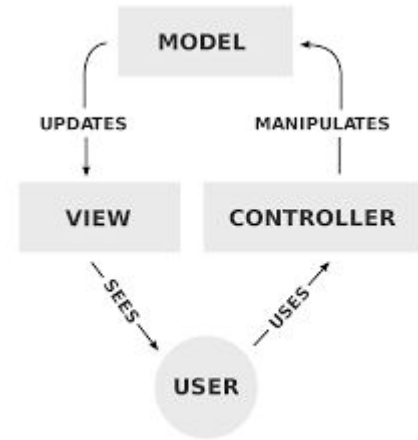
6. rest of semester



Model-View-Controller (MVC)

MVC is usually described in terms of user-facing GUI apps, but also applies to services and other software

- Model contains the core functionality and data
- View provides information to a client via an API, potentially with different APIs - facades - for different views
- Controller handles client input entered through the view's API and directs the model to "do something"



MVC Analogy

In a restaurant, the Chef is the Model. It's job is to take orders (requests) from the Waiter and create food (responses) for the Waiter to deliver to the Customer



A Waiter is a Controller. Waiters take in orders (from clients or users), take them back to the Chef (Model), and then deliver the proper meal to Customers (clients or users)

The Table is the View. It just holds (makes available or displays) whatever meal the Waiter brings back from the Chef

There can be multiple controllers (waiters) and views (tables) for the same model (chef)



Model

Includes business logic that manipulates state and does most of the work as well as CRUD operations for persistent state (application or domain data). Ideally a fully functional system

- Not possible to alter data except through the model
- Provides APIs to views and controllers that enable the outside world to communicate with it
- Responds to requests for information about its state (usually from the view, possibly through the controller)
- Responds to instructions to do something or change state (from the controller)



View and/or Controller may include some persistent state, such as saved layout and preferences, but the Model *owns* the domain data

View

Updates when model changes, acts as passive observer that does not affect the model

- May be multiple views of same data, e.g., multiple APIs
- Push - view registers callbacks with model, model notifies of changes
- Pull - view polls model for changes

Remember, MVC views do not necessarily involve a user interface, a view could be the representation "visible" to other code, e.g., for that other code to call through a view-specific API



Controller

Accepts inputs or calls from client, translates actions on view into operations on model, decides what model should do

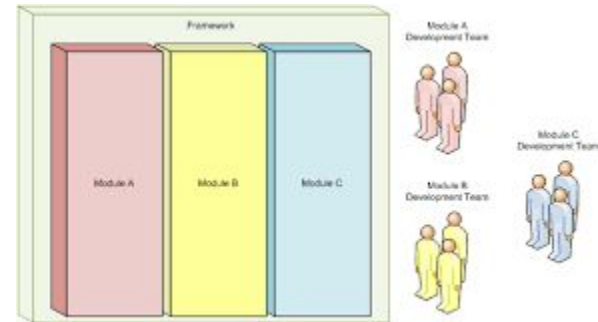
- May be structured as intermediary between view and model
- May be multiple controllers for same model, e.g., for different clients



“Separation of Concerns”

Important software engineering concept that you will keep hearing about

- Model is loosely coupled with the view and controller, knows nothing about either ⇒ *modularity*
- However, view and controller may be tightly coupled
- Usually implemented as two or three different classes or modules that can be developed separately



Example: music player

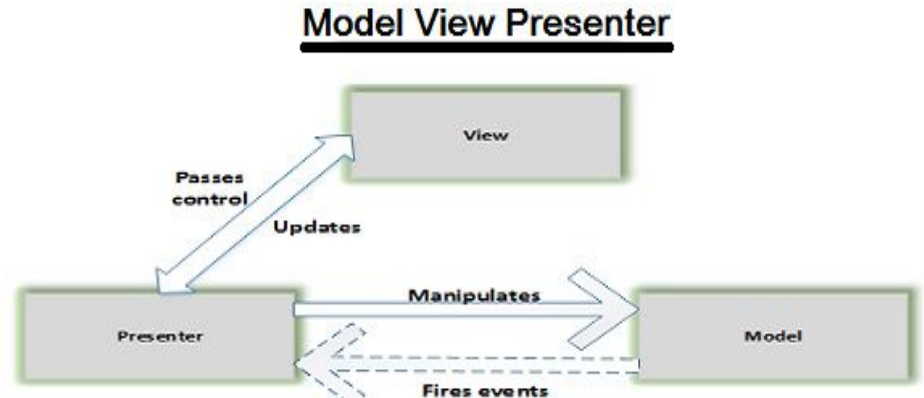
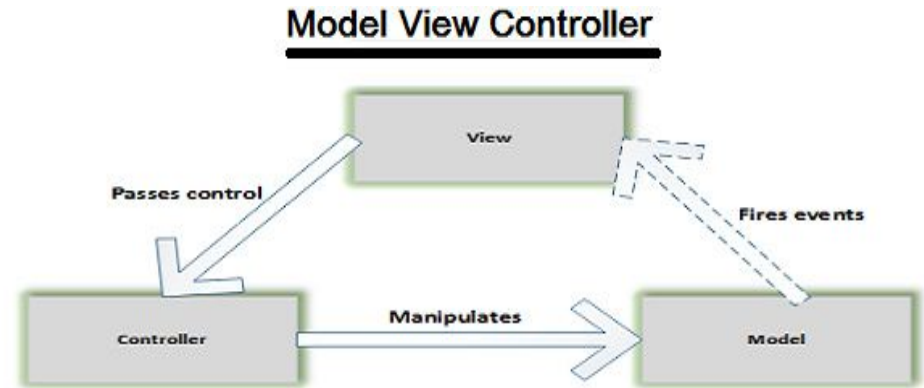
- View - playlists, current song, possibly graphics and video as well as audio, with some elements that accept selection, action, data entry
- Controller - interprets the actions applied to view and tells model what to do
- Model - stores content, knows how to play songs, shuffle, rip, and so on



MVC Variants

Controversy over what belongs in the model vs. other parts

The model might be *only* the data (except possibly for some bookkeeping methods) and the business logic resides in the controller (which better matches [N-tier architecture](#))



How to decide?

Business logic belongs in the model when the data, and thus the model, is only part of one program

But the developer may not know in advance whether the model might eventually be used for other programs
([software sustainability](#))



Business logic belongs in the controller when the data, and thus the model, is shared across many programs

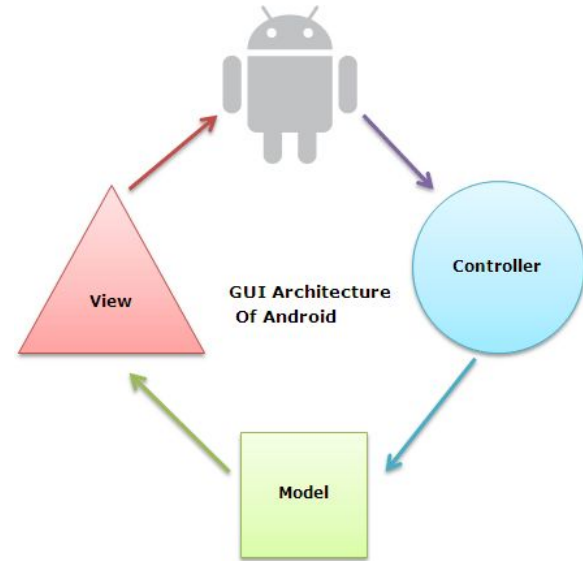
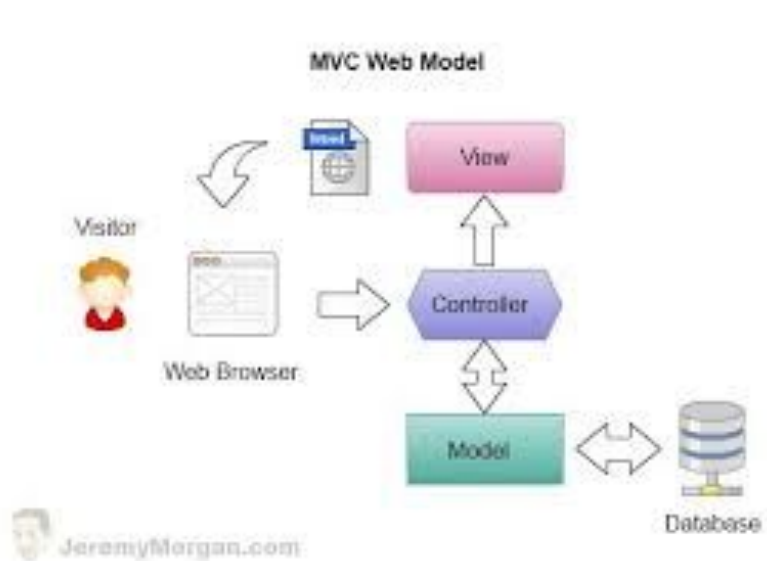
Different controller (and presumably different view or set of views) for each program

Enables replacing model while retaining controller (and views)

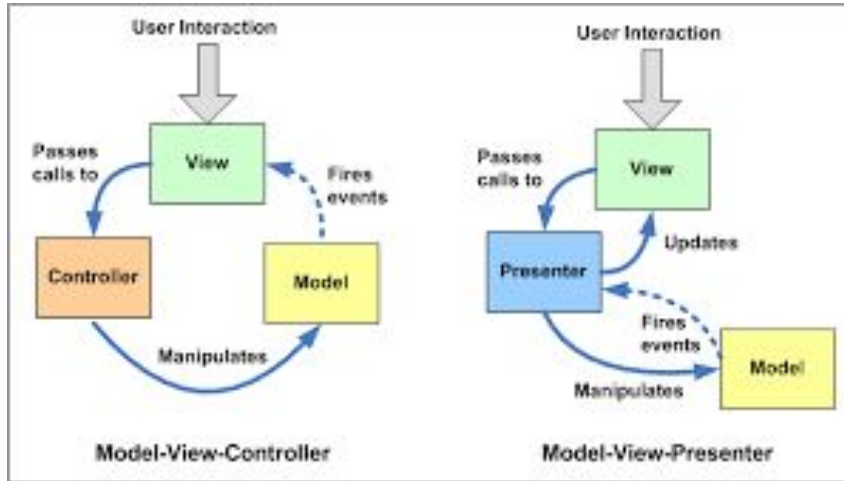
Is MVC really a design pattern?

It's not in the GoF book

But it works and everyone uses it (or something like it)



Is MVC a design pattern?



MVC - uses Observer Pattern to passively update view

MVP - uses Observer Pattern to passively update presenter, which in turn actively updates the view

([There are other variants](#))

MVC/P is a *Compound* Design Pattern

Observer Pattern + Strategy Pattern + Facade Pattern (when applied to code clients)

Observer Pattern updates the view, possibly multiple different views

Strategy Pattern supports different behaviors (“strategies”) that leverage the model, e.g., multiple clients using the same service

Facade Pattern enables multiple variants or subsets of the model's API



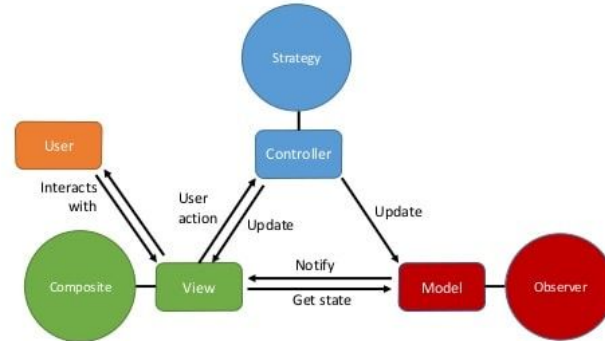
MVC Code Examples

C++

Java

warning: most MVC code examples
and frameworks assume GUI apps

Traditional MVC & associated design patterns



1. R. Buencamino: Learning the Model-View-Controller Design Patterns in iOS. Lynda.com, 2015

Agenda

1. authentication/authorization demo: Shivam Chaturvedi and Garlic Bread team
2. MVC

as time permits:

3. **test oracles**
4. test suite quality
5. test generation
6. rest of semester



What is a Test Oracle?

An entity that already knows the expected outputs for given inputs and/or can check that the actual outputs are the same as or consistent with the expected outputs



Often the human tester is the test oracle - either they know the correct output in advance or, given an actual output, can easily figure out whether or not it is correct

For manual testing, the human tester checks the results

For automated testing, the human tester writes assertions to automatically check the results

How Does the Human Tester Know the Correct Outputs?

If it doesn't crash, hang, or produce obviously nonsensical output, it works!

Stick to math and string processing libraries, with well-understood functions

Wait until users complain that the software doesn't do what they expected, then write test cases to check for what they say they expected



Prose Specifications

Detailed prose specification - common for networking, database, file system and other [standard protocols](#) where interoperability is a primary concern, rare for user-facing functionality of business/consumer applications

[\[Search\]](#) [\[txt\]](#) [\[html\]](#) [\[pdf\]](#) [\[with errata\]](#) [\[bibtex\]](#) [\[Tracker\]](#) [\[WG\]](#) [\[Email\]](#) [\[Diff1\]](#) [\[Diff2\]](#) [\[Nits\]](#)

From: [draft-ietf-uri-ur1-07](#)

Obsoleted by: [4248](#), [4266](#)

Updated by: [1808](#), [2368](#), [2396](#), [3986](#), [6196](#), [6270](#),
[8089](#)

Network Working Group

Request for Comments: 1738

Category: Standards Track

Proposed Standard

[Errata exist](#)

T. Berners-Lee

CERN

L. Masinter

Xerox Corporation

M. McCahill

University of Minnesota

Editors

December 1994

Uniform Resource Locators (URL)

Status of this Memo

This document specifies an Internet standards track protocol for the Internet community, and requests discussion and suggestions for improvements. Please refer to the current edition of the "Internet Official Protocol Standards" (STD 1) for the standardization state and status of this protocol. Distribution of this memo is unlimited.

Abstract

This document specifies a Uniform Resource Locator (URL), the syntax and semantics of formalized information for location and access of resources via the Internet.

1. Introduction

This document describes the syntax and semantics for a compact string representation for a resource available via the Internet. These strings are called "Uniform Resource Locators" (URLs).

The specification is derived from concepts introduced by the World-Wide Web global information initiative, whose use of such objects dates from 1990 and is described in "Universal Resource Identifiers in WWW", [RFC 1630](#). The specification of URLs is designed to meet the requirements laid out in "Functional Requirements for Internet Resource Locators" [\[12\]](#).

This document was written by the URI working group of the Internet Engineering Task Force. Comments may be addressed to the editors, or to the URI-WG uri@bunyip.com. Discussions of the group are archived at [URL:http://www.acl.lanl.gov/URI/archive/uri-archive.index.html](http://www.acl.lanl.gov/URI/archive/uri-archive.index.html)

Formal Specifications

Expensive to write and verify, mostly limited to safety-critical systems

$$\frac{P \rightarrow P' \quad \{P'\} C \{Q'\} \quad Q \rightarrow Q'}{\{P\} C \{Q\}}_{\text{WEAK}}$$

$$\frac{}{\{A[E/x]\} x := E \{A\}}_{\text{ASG}}$$

$$\frac{\{P\} C_1 \{A\} \quad \{A\} C_2 \{Q\}}{\{P\} C_1; C_2 \{Q\}}_{\text{SEQ}}$$

$$\frac{\{P \wedge B\} C_1 \{Q\} \quad \{P \wedge \neg B\} C_2 \{Q\}}{\{P\} \text{ if } B \text{ then } C_1 \text{ else } C_2 \{Q\}}_{\text{IF}}$$

$$\frac{\{I \wedge B\} C \{I\}}{\{I\} \text{ while } B \text{ C } \{I \wedge \neg B\}}_{\text{While}}$$



Other than standard protocols and safety-critical systems, "specifications" are usually informal user stories and/or use cases that leave test oracles up to developers

Sometimes There Isn't An Agreed Upon Test Oracle

Consider this search: <https://www.google.com/search?q=gail+kaiser>

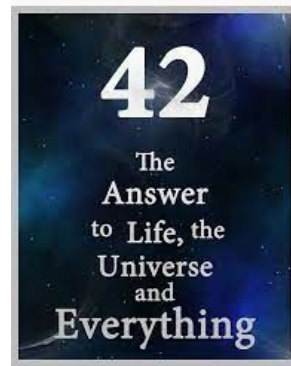
Is the answer correct?

Are the answers on the first page the best ones?

Now consider this search: [gail kaiser - Search \(bing.com\)](https://www.bing.com/search?q=gail+kaiser)

Does it give the same links in the same order?

[Google](#) and [Microsoft](#) put a lot of time and effort into rigorously testing their search engines, but their search engine results do not always agree

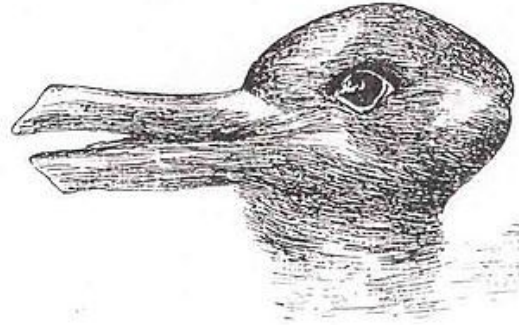


Sometimes There Isn't Any Test Oracle

Write a program that determines [exactly how many stars there are in the universe](#)

Write a program that correctly labels [ambiguous images](#)

Write a program that calculates when the Earth will [run out of fossil fuels](#)



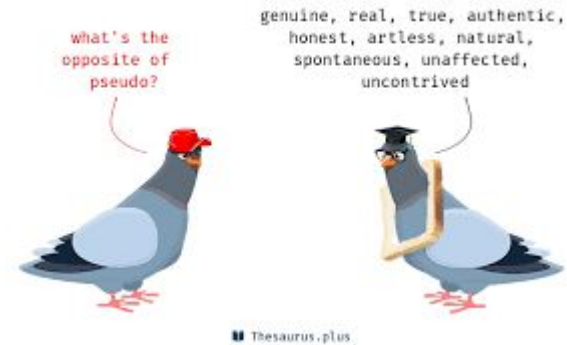
Non-testable program = *“Programs which were written in order to determine the answer in the first place. There would be no need to write such programs, if the correct answer were known.”* ([Weyuker 1982](#))

Pseudo-Oracles

Pseudo-oracle = one test case acts as an oracle for another test case

You may not know whether the result of either test case is right, but if the results are inconsistent you know (at least) one of them is wrong

They could both be wrong



Pseudo-Oracles: Regression Testing

Regression testing is the most common form of pseudo-oracle

The test results for the old version are known and we re-run the tests on the new version to see if they have the same or different results

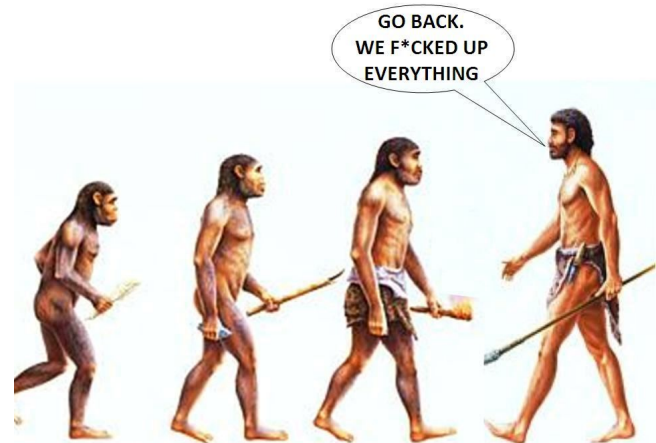
Some test cases *should* change results on the new version $\Rightarrow$ when we try to fix a bug that caused a test case to fail on the old version, we want that test case to pass on the new version

But unrelated test cases should not be affected $\Rightarrow$ if they are, that indicates a hidden dependency and (probably) a newly introduced bug

`program_v1(input) -> output_v1`

`program_v2(input) -> output_v2`

`output_v1 =? output_v2`



Pseudo-Oracles: Differential Testing

Differential testing requires a second *independently developed* implementation of the same functionality, as well as a test inputs applicable to both

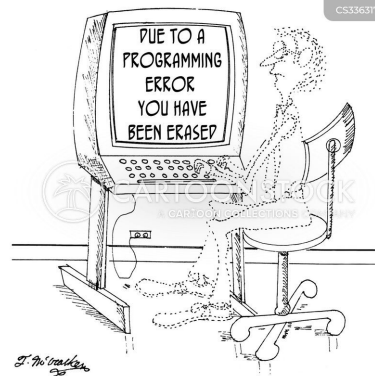
Practical for standard protocols, e.g., there are many implementations of SSL/TLS and HTTP

- If they disagree, one or both is wrong
- If they agree, that does not guarantee both are right, but does provide some confidence that they are both right

```
program_1(input) -> output_1
```

```
program_2(input) -> output_2
```

```
output_1 ==? output_2
```



It's critical that the implementations are truly independent, with no common third-party libraries or other common factors

But “independent” developers can make the same mistakes

Pseudo-Oracles: Metamorphic Testing

[Metamorphic testing](#) requires only one implementation

Starts with some original input and its known original output. We do not need to know whether this output is correct

Create a new test case by *deriving* a new input from the original input, and *predicting* the expected new output from the original input, the derived input and the original output - the prediction is usually that the output is the same, nearly the same, or changed in a simple way

If the actual new output deviates too much from the predicted output, there is (probably) a bug

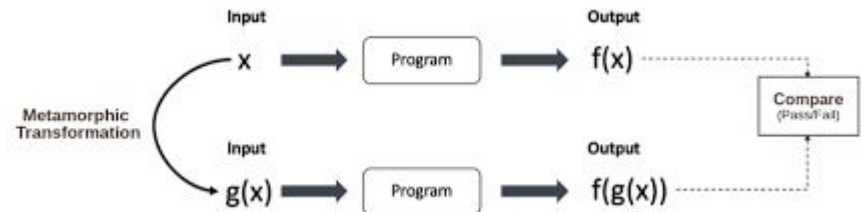
```
program(input_1) -> output_1
```

```
deriveInput(input_1) -> input_2
```

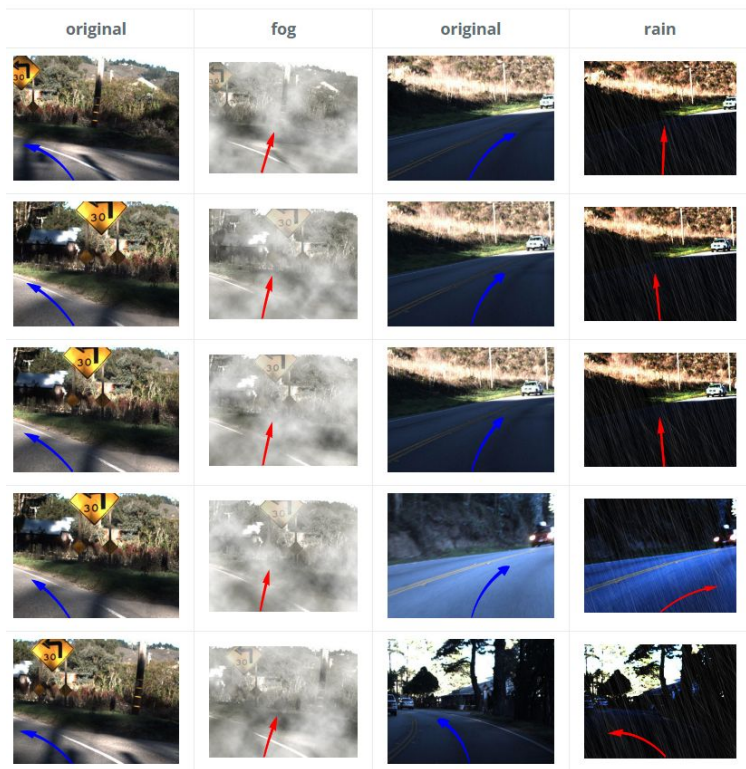
```
predictOutput(input_1, input_2,  
output_1) -> predicted_output_2
```

```
program(input_2) -> output_2
```

```
check(predicted_output_2, output_2)  
-> do they match?
```



Example Metamorphic Testing



Say we have a machine learning application that controls the steering of a [self-driving car](#), so it obeys traffic laws and doesn't cause accidents

How do we know what is “correct” steering for a given real-world driving scenario? How can we write test cases?

Different human drivers will not necessarily steer at exactly the same angle and the same human driver will not necessarily steer at the same angle every time

[DeepTest](#): Synthetic but realistic changes to driving images such as adding snow, fog, rain, more sunlight, less sunlight, etc. should not (in most cases) change automobile steering

Predicted Output is not Necessarily the Same Output

Metamorphic relations for a sorting program (lowest first)
applied to an array of numbers

`sort(5,2,3,1,4) -> 1,2,3,4,5`

Permutative: if we shuffle the order of the input array, the
sorted output array should be the same

`sort(3,4,1,2,5) -> 1,2,3,4,5` 😊

Additive: if we add N to every element of the input array,
the sorted output array should be in the same order

`sort(25,22,23,21,24) -> 21,22,23,24,25` 😊

Multiplicative: if we multiply every element of the input
array by positive N, the sorted output array should be in
the same order

`sort(15,6,9,3,12) -> 3,6,9,12,15` 😊

Invertive: if we multiply every element of the input array
by negative N, the sorted output array should be in the
opposite order

`sort(-5,-2,-3,-1,-4) -> -5,-4,-3,-2,-1` 😊

Inclusive: if we add one new element to the input array,
the sorted output array should be the same except for
the placement of that new element

`sort(5,2,42,3,1,4) -> 1,2,3,4,5,42` 😊

Exclusive: if we remove one element from the input
array, the sorted output array should be the same except
it is missing that one element

`sort(5,3,1,4) -> 1,3,4,5` 😊

Using Metamorphic Relations to Detect Bugs

Original

`buggysort(5,2,3,1,4) -> 1,2,3,4,5`

Permutative: if we shuffle the order of the input array, the sorted output array should be the same

`buggysort(3,4,1,2,5) -> 2,4,1,5,3` 😞

Additive: if we add N to every element of the input array, the sorted output array should be in the same order

`buggysort(25,22,23,21,24) -> 25,25,25,25,25` 😲

Multiplicative: if we multiply every element of the input array by (positive) N, the sorted output array should be in the same order

`buggysort(15,6,9,3,12) -> 6,12,3,15,9` 😞

Invertive: if we multiply every element of the input array by (negative) N, the sorted output array should be in the opposite order

`buggysort(-5,-2,-3,-1,-4) -> -1,-4,-3,-2,-5` 😞

Inclusive: if we add one new element to the input array, the sorted output array should be the same except for the placement of that new element

`buggysort(5,2,42,3,1,4) ->` ⌚ 😡

Exclusive: if we remove one element from the input array, the sorted output array should be the same except it is missing that one element

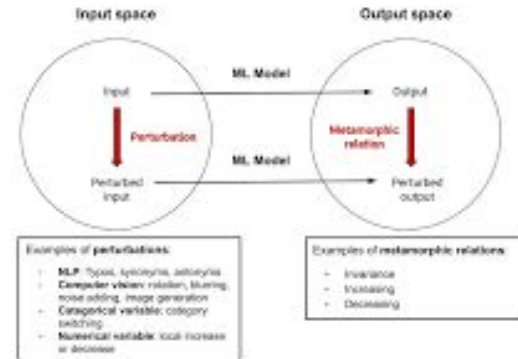
`buggysort(5,3,1,4) ->` 🌋 😲

Where Do Metamorphic Relations Come From?

Some are generic, e.g., permutation can be applied to any ordered input

Others are specific to domain or application such as car steering

Metamorphic Testing automates generation of additional tests from existing tests whether or not there is a test oracle - it's a test generation technique as well as a pseudo-oracle technique



Agenda

1. authentication/authorization demo: Shivam Chaturvedi and Garlic Bread team
2. MVC

as time permits:

3. test oracles
4. test suite quality
5. test generation
6. rest of semester



Test Suite Quality

An empty test suite has 0% coverage but all its tests pass!

A non-empty test suite consisting entirely of valid inputs, where all the tests pass, may have 100% coverage but low quality because the code is not checking for invalid inputs

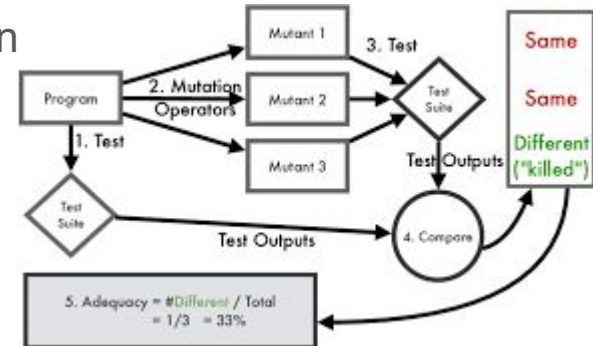
Remember no test suite can cover code that has not been written



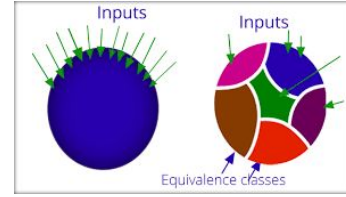
Testing the Test Suite

How do we know whether we have a “good” or “good enough” test suite?

- ✓ [Equivalence Partitioning \(and Boundary Analysis\)](#) - for outputs as well as inputs
- ✓ Control Flow Coverage - high percentage of statement, branch, [MC/DC](#) & [MCC](#)
- ✓ [Data Flow Coverage](#) - high percentage of intraprocedural paths from each variable or field definition (write) to everywhere the written value is used (read) before it is overwritten
- ✓ [Mutation Testing](#) - use mutation operators to inject bugs into the code (e.g., switch > to <, insert return statements at arbitrary locations in the middle of functions) and run the regular test suite, did it “kill” (detect) all the mutants?



Equivalence Partitioning for *Inputs*



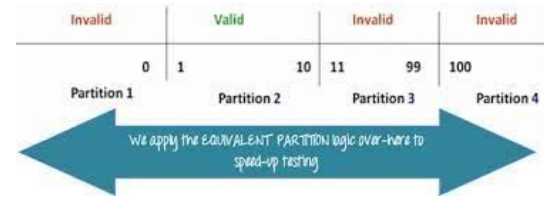
Inputs include any parameters that are read, any global in-memory state that is read, any environment data that is read (from files, databases, network, devices, environment variables, etc.), *and the return values and other shared state received from API, library and system calls that are read* (used for anything) by the code under test.

Equivalence partitioning of parameters is not enough, also need to test input equivalence partitions for global variables, environment data, and outputs from any external calls that are, effectively, inputs to the code under test. Forcing some of these equivalence partitions may require mocks.

If any of these input values is a status code, need to test with all possible status codes from that source.

Have both valid and invalid input partitions been tested? Valid vs. invalid is wrt *your* code, not necessarily the code it comes from.

Equivalence Partitioning for *Outputs*



Outputs include any (object, by-reference or data structure) parameters that are written, any global in-memory state that is written, any environment data that is written (files, databases, network, devices, environment variables, etc.), *and the parameters and shared state sent to API, library and system calls* by the code under test.

Equivalence partitioning of return values is not enough, also need to test output equivalence partitions for updated parameters, global variables, environment data, and inputs to any external calls. Forcing output equivalence partitions typically requires specially crafted inputs that lead to generating those outputs.

If any of these output values is a status code, need tests that force all possible status codes.

If it's possible for the code under test to produce invalid outputs, then invalid output equivalence partitions need to be covered in addition to valid output partitions.

For many of these cases, we may not know what code might eventually receive these outputs.

Control Flow Coverage

Multiple Condition Coverage (MCC) and
Modified Condition/Decision Coverage (MC/DC)
rarely used outside safety-critical software

MCDC: each individual condition is true at least
once and false at least once [linear]

MCC: conditions must evaluate to T or F in all
combinations [combinatoric]

Short circuit: Only need to test conditions that
independently affect decision's outcome

(false AND anything-else)

```
if (A and B)
  { do something }
else
  { do something else }
```

```
if (A or B)
  { do something }
else
  { do something else }
```

Example

```
boolean purchaseAlcohol  
(int buyerAge, int ageFriend)  
{  
    boolean allow = False;  
    if ((buyerAge>=21) or  
        (ageFriend>=21))  
    { allow = True; }  
    return allow;  
}
```

Assert purchaseAlcohol(25,25) == True
gives 100% *statement* coverage

Assert purchaseAlcohol(25,25) == True
Assert purchaseAlcohol(20,20) == False
gives 100% *branch* coverage

But *both* the buyer *and* the friend must be
21 or older

Does MCDC coverage detect the bug?

Example continued

```
boolean purchaseAlcohol  
(int buyerAge, int ageFriend)  
{  
    boolean allow = False;  
    if ((buyerAge>=21) or  
        (ageFriend>=21))  
    { allow = True; }  
    return allow;  
}
```

purchaseAlcohol(25,25) == True covers T, T
purchaseAlcohol(20,20) == False covers F, F
purchaseAlcohol(25,20) == False covers T, F
purchaseAlcohol(20,25) == False covers F, T

Does a set of test cases fulfilling MCC
coverage (truth table) detect the bug?

Data Flow Coverage

`x := 12345 ;`

lots of code that does not change x

`y := derive some value from x ;`

some code that changes x

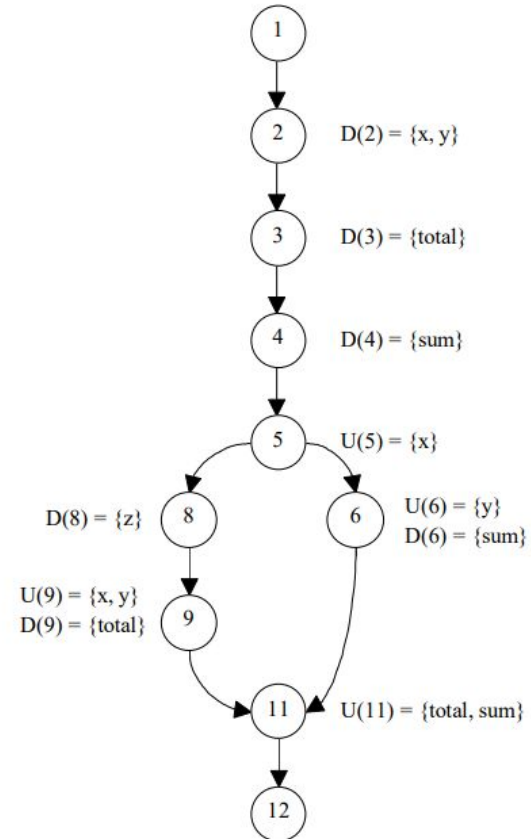
`z := derive some value from x;`

Cover all paths from a variable definition (write) to any uses (reads) of that variable *before* its value is overwritten = a new definition

Data flow is about reads and writes, not variable declarations

Example

```
1. begin
2.   read ( x, y );
3.   total := 0.0;
4.   sum := 0.0;
5.   if x <= 1
6.     then sum := y
7.     else begin
8.       read (z);
9.       total := x * y;
10.    end;
11.   write (total, sum);
12. end.
```

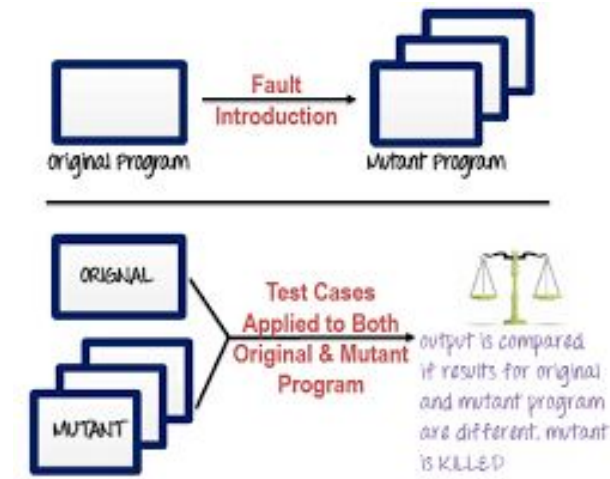


Mutation Testing

Intuition: Does your test suite actually detect bugs? Check by changing your code to intentionally insert a bug, then run your test suite. Did any test that passed with the original code and exercises the changed code now fail with the modified code? If not, then your test suite is not good enough to detect the bug you inserted. Repeat with another bug, etc.

Example mutation testing tools:

[PIT](#) for Java, [Mull](#) for C and C++



Mutation Testing Terminology

Mutation Analysis = another name for Mutation Testing

Mutant = small change in one method to insert a bug; otherwise same codebase and same tests

Mutation Score = number of mutants killed / total number of non-equivalent mutants

Equivalent Mutants = functionally equivalent to the original program, or to some other mutant generated using a different mutation operator and/or mutating a different method; remove all equivalent mutants before testing with the rest of the mutants to compute mutation score



Mutation Operators

Statement deletion

Statement duplication or insertion, e.g., `return 0;`

Replacement of boolean subexpressions with *true*, *false*

Replacement of some arithmetic operations with others, e.g., `+` with `*`, `-` with `/`

Replacement of some boolean relations with others, e.g., `>` with `>=`, `==` and `<=`

Replacement of variables with others from the same scope (variable types must be compatible)

Changes in synchronization concerned with concurrency and multi-threading

Insert Security Vulnerability (from [CVEs](#) and [CWEs](#))



Original Method		With Embedded Mutants
<pre>int Min (int A, int B) { int minVal; minVal = A; if (B < A) { minVal = B; } return (minVal); } // end Min</pre>		<pre>int Min (int A, int B) { int minVal; minVal = A; minVal = B; if (B < A) if (B > A) if (B < minVal) { minVal = B; Bomb(); minVal = A; minVal = failOnZero (B); } return (minVal); } // end Min</pre>
	Δ1	
	Δ2	
	Δ3	
	Δ4	
	Δ5	
	Δ6	

Mutation Testing Relies On...

- “Competent Programmer Hypothesis”: Most software faults introduced by experienced programmers are due to small errors
- “Coupling Effect Hypothesis”: Simple faults can cascade to form emergent faults
- “RIP Model”:
 1. A test must *reach* the mutated statement
 2. Test input data should *infect* the program state by causing different program states for the mutant vs. the original program
 3. The incorrect program state must *propagate* to the program's (or method's) output so it can be checked by the test

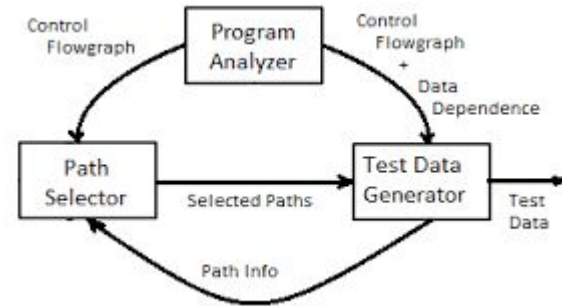


Agenda

1. authentication/authorization demo: Shivam Chaturvedi and Garlic Bread team
2. MVC

as time permits:

3. test oracles
4. test suite quality
5. **test generation**
6. rest of semester



Fuzzing (aka Fuzz Testing)

Start with a “seed” input and semi-randomly mutate it (or parts of it), intending to construct malformed or invalid input



Normally applied to relatively long inputs or sequences of inputs like packets, files, and protocols, not to primitive types like integers and strings except as they appear in the fields of larger structured content

Fuzzer leverages mandatory features and constraints of data types and metadata formats, e.g., required fields set to zero length, metadata says the contents are version 3 format but the contents are version 1 (and incompatible with version 3)

Fuzzers usually do not know anything about program semantics, so assertions limited to program crashes, program is very slow, program uses too much memory, etc.

Fuzzing



Fuzz Testing is mostly used to detect security vulnerabilities, so generation tries to produce illegal inputs that make it past any input checkers, validators or sanitizers that typically operate at entry points. *Unchecked* invalid input is how hackers get past defenses and trick software into processing their specially crafted data (“attack vector”)

Feedback/guidance encourages generating new inputs that exercise longer execution paths that reach further into the target software - e.g., after the fuzzer finds a malformed packet header that is accepted by the program, it keeps using that header while fuzzing different parts of the packet body

Easy to implement and use, and yields lots of test inputs - but requires some existing test inputs as seeds

examples: [AFL++](#), [OSS-Fuzz](#)

Free Fuzzing Book

[The Fuzzing Book](#)

From the same lead author as free [The Debugging Book](#)



Automated Test Generation

Metamorphic testing generates both the test input and the assertion (“is it consistent with the predicted output?”) - although it requires some existing inputs to derive from and thus is limited by the quality of the original test suite

Regression, differential and fuzz testing also require an existing test suite

But ideally test generation tools could augment even poor quality or non-existent manually written test cases with high quality test suites



Most Test Generators are Input Generators

The best most test generation techniques can do is generate inputs that are useful for triggering bugs, which is hard, but generating high quality assertions that can accurately detect triggered bugs can be even harder

The assertions typically must be written by human developers OR easy to check generic assertions are used, like did it store the input value, did it crash?

Some test generators assume the current code is correct (🧐), but bugs may be inserted later, so the results of testing the current code are recorded and those results are inserted into the assertions used during regression testing



Random Test Generation

Selects inputs at random from the input space

The test generator need not know anything about equivalence partitioning except for built-in partitions of primitive types, e.g., MinInt, negative, zero, positive, MaxInt

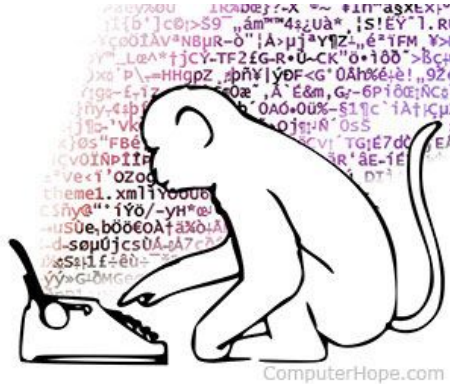
Possible lengths of strings and arrays might be determined by conventional data sizes: empty, one element, four elements, eight elements, 16, 32, 64, 128, 256, etc. elements (stop growing when code under test starts failing)

How is random different from fuzz? Fuzzing is usually for changing small parts of otherwise valid structured inputs, random is usually for changing entire inputs



Example

```
int myAbs(int x) {  
    if (x > 0) {  
        return x;  
    }  
    else {  
        return x; // bug: should be '-x'  
    }  
}
```



The random tests for this function could be {123, 36, -35, 48, 0}. Of these, only the value '-35' triggers the bug

Most random testing techniques aren't truly random, they use some feedback/guidance mechanism to avoid redundancy and to focus on inputs likely to find bugs

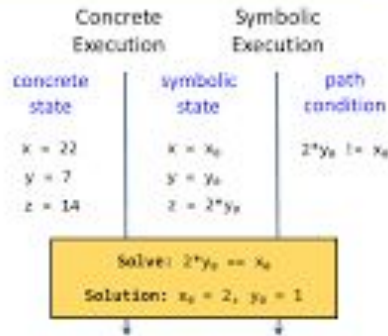
Easy to implement and use, yields lots of test inputs, but not necessarily a "high quality test suite"

example: [Randoop](#)

Symbolic Execution

An Illustrative Example

```
int foo(int v) {  
    return 2*v;  
}  
  
void test_me(int x, int y) {  
    int z = foo(y);  
    if (z == x)  
        if (x > y+10)  
            ERROR;  
}
```



Generalizes testing by executing the program abstractly, so one abstract execution covers multiple possible inputs of the program that share a particular execution path through the code

Useful for increasing coverage: execute symbolically from a program entry point until execution reaches an uncovered branch, then use constraint solving on the path condition to identify at least one set of inputs that will force executing that path. Construct a conventional test case with those inputs

Example where g is path condition and E is symbolic environment

1 int x=0, y=0, z=0;

2 if(a) {

3 x = -2;

4 }

5 if (b < 5) {

6 if (!a && c) { y = 1; }

7 z = 2;

8 }

9 assert(x + y + z != 3);

line	g	E
0	true	$a \mapsto \alpha, b \mapsto \beta, c \mapsto \gamma$
1	true	$\dots, x \mapsto 0, y \mapsto 0, z \mapsto 0$
2	$\neg \alpha$	$\dots, x \mapsto 0, y \mapsto 0, z \mapsto 0$
5	$\neg \alpha \wedge \beta \geq 5$	$\dots, x \mapsto 0, y \mapsto 0, z \mapsto 0$
9	$\neg \alpha \wedge \beta \geq 5 \wedge 0 + 0 + 0 \neq 3$	$\dots, x \mapsto 0, y \mapsto 0, z \mapsto 0$

line	g	E
0	true	$a \mapsto \alpha, b \mapsto \beta, c \mapsto \gamma$
1	true	$\dots, x \mapsto 0, y \mapsto 0, z \mapsto 0$
2	$\neg \alpha$	$\dots, x \mapsto 0, y \mapsto 0, z \mapsto 0$
5	$\neg \alpha \wedge \beta < 5$	$\dots, x \mapsto 0, y \mapsto 0, z \mapsto 0$
6	$\neg \alpha \wedge \beta < 5 \wedge \gamma$	$\dots, x \mapsto 0, y \mapsto 1, z \mapsto 0$
6	$\neg \alpha \wedge \beta < 5 \wedge \gamma$	$\dots, x \mapsto 0, y \mapsto 1, z \mapsto 2$
9	$\neg \alpha \wedge \beta < 5 \wedge \neg(0 + 1 + 2 \neq 3)$	$\dots, x \mapsto 0, y \mapsto 1, z \mapsto 2$

Concolic Execution

“Concolic execution” is a special case of symbolic execution where some of the inputs are concrete (specific known values) and some are symbolic (α , β , γ , $\$thisIsAnInteger$)

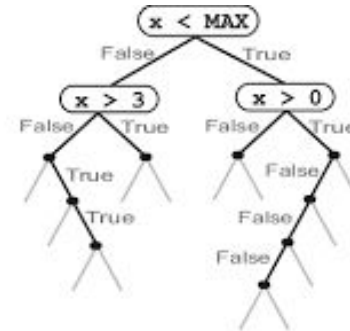
Symbolic and concolic execution are not easy to implement and use, and do not scale well to programs with high cyclomatic complexity due to [path explosion](#)

What is [cyclomatic complexity](#)?

Number of decision points (if, for, while, etc.) plus 1. A function with no conditions has only a single possible execution path = 1

examples: [Klee](#), [JavaPathFinder](#)

```
void write( int x )  
{  
  if (x < MAX) {  
    if (x > 0)  
      ...  
    else  
      ...  
  } else {  
    if (x > 3)  
      ...  
    else  
      ...  
  }  
}
```



Agenda

1. authentication/authorization demo: Shivam Chaturvedi and Garlic Bread team
2. MVC

as time permits:

3. test oracles
4. test suite quality
5. test generation

6. rest of semester



Rest of Semester

Second Iteration due November 25 (**next week on Monday!**)

Second Iteration Demo due December 6
(Friday of the week after Thanksgiving)

Second Assessment Friday December 6
12:01AM to Monday December 9
11:59PM (last day of classes and the day
before university "study days")

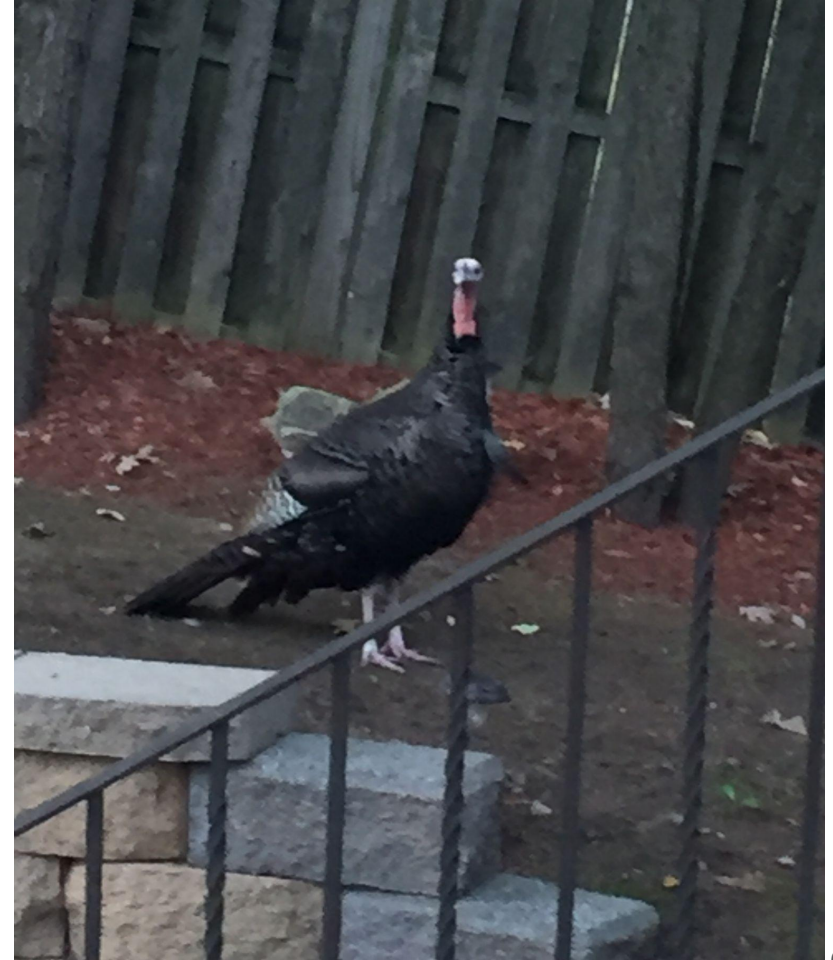
Demo Day Monday December 16 10AM to
4PM (Monday of finals week)



Next Week

Tuesday Class Cancelled

Happy Thanksgiving



Ask Me Anything

